FUNDACIÓN ESCUELA TECNOLÓGICA DE NEIVA
JESÚS OVIEDO PÉREZ

ARQUISELECT: SISTEMA EXPERTO PARA LA SELECCIÓN DE ESTILOS DE ARQUITECTURA DE SOFTWARE

Nicolás Andrés Chica Córdoba

Juan Diego Bonilla Aroca

Tutor

Juan José Ramos Diaz

Fundación Escuela Tecnológica de Neiva

Ingeniería de software

Rivera-Huila

2026

FUNDACIÓN ESCUELA TECNOLÓGICA DE NEIVA
JESÚS OVIEDO PÉREZ

Título del Proyecto

ArquiSelect: Sistema Experto para la Selección de Estilos de Arquitectura de Software basado en Reglas Lógicas, Priorización de Atributos de Calidad y Explicación del Razonamiento, implementado en Prolog e integrado con Python**.**

Descripción del Problema

La arquitectura de software es el conjunto de decisiones estructurales que determinan cómo se organizará un sistema, cómo crecerá y cuánto costará mantenerlo. Una de las primeras y más importantes de esas decisiones es la elección del estilo arquitectónico: monolito en capas, monolito modular, microservicios, arquitectura orientada a eventos o serverless. Cada estilo ofrece ventajas en ciertos escenarios y genera costos considerables en otros, por lo que no existe una opción que sea la mejor en todos los casos.

En la práctica, esta decisión suele tomarse por intuición, por moda tecnológica o por imitación de lo que usan empresas grandes. Es común ver equipos de dos o tres desarrolladores que adoptan microservicios para una aplicación con pocos usuarios, y terminan con una complejidad de despliegue, monitoreo y comunicación entre servicios que no pueden sostener. También ocurre lo contrario: sistemas con alta demanda y requisitos de disponibilidad que se construyen como un monolito rígido, y que después resultan costosos de escalar. En ambos casos el error no proviene de la falta de herramientas, sino de la falta de criterio para relacionar los requisitos del proyecto con las características de cada estilo.

Este problema se agrava en contextos académicos y regionales, como el de los estudiantes y desarrolladores junior de Neiva y el Huila, donde los equipos son pequeños, los presupuestos son limitados y no siempre se cuenta con un arquitecto con experiencia que oriente la decisión.

A esto se suma que el conocimiento sobre qué estilo conviene en cada situación está disperso en libros, artículos y en la experiencia de profesionales, y casi nunca está organizado de forma consultable.

El sistema propuesto busca resolver esta situación. A partir de las condiciones del proyecto que el usuario describa (tamaño del equipo, presupuesto, escalabilidad esperada, nivel de disponibilidad, complejidad del dominio, tiempo de entrega y experiencia en despliegue), el sistema recomendará un estilo arquitectónico, indicará alternativas ordenadas por conveniencia y explicará con qué reglas llegó a esa conclusión. Los usuarios finales serían:

- **Estudiantes de ingeniería de software** que necesitan justificar la arquitectura de sus proyectos de curso.
- **Desarrolladores juniors y equipos pequeños** que deben decidir una arquitectura sin un arquitecto senior disponible.
- **Emprendedores y pequeñas empresas de software** que quieren evitar sobrecostos por elegir una arquitectura inadecuada.
- **Docentes**, que pueden usar el sistema como herramienta de apoyo para discutir los criterios de decisión en clase.

Justificación como Sistema Experto

Seleccionar una arquitectura no es un cálculo con una fórmula única, sino un proceso de razonamiento propio de un experto. Un arquitecto experimentado no consulta una tabla fija: relaciona los requisitos con los atributos de calidad que realmente importan, evalúa qué estilos los favorecen, descarta los que son incompatibles con las restricciones y pondera los que quedan. Ese proceso se puede representar mediante reglas, lo que hace que el problema sea apropiado para un sistema experto por las siguientes razones.

Razonamiento basado en reglas. El conocimiento del dominio se expresa de forma natural como reglas condicionales. Por ejemplo, si el equipo es pequeño y el presupuesto es bajo, los microservicios deben descartarse; si la carga es muy variable e impredecible, la arquitectura serverless gana puntos. Estas reglas se escriben como hechos y cláusulas en Prolog, y pueden ser leídas, corregidas y ampliadas por un experto sin modificar el resto del programa.

Encadenamiento lógico. La decisión no se toma en un solo paso, sino en varios niveles encadenados: los requisitos del proyecto determinan los atributos de calidad prioritarios, los atributos de calidad determinan qué tácticas arquitectónicas se necesitan, y esas tácticas favorecen o penalizan a cada estilo. El motor de inferencia de Prolog, mediante resolución SLD y retroceso (backtracking), recorre esta cadena de forma automática y permite además reconstruir el camino seguido para dar una explicación.

Manejo de prioridades e incertidumbre. Los requisitos suelen entrar en conflicto: la escalabilidad empuja hacia microservicios, mientras que un equipo pequeño y un presupuesto bajo empujan hacia un monolito modular. Por eso la respuesta no es un simple sí o no, sino una puntuación ponderada. A cada regla se le asigna un peso según su importancia, y las reglas de exclusión (con negación como fallo) eliminan estilos inviables. Cuando la información del usuario es incompleta o ambigua, el sistema trabaja con valores por defecto y reporta un nivel de confianza menor en su recomendación.

Diferencia frente al software convencional. Un programa convencional resolvería el problema con una cadena de condicionales dentro del código, donde la lógica del dominio queda mezclada con la lógica del programa. Cualquier cambio en el conocimiento obliga a modificar y volver a compilar el sistema, y el programa solo entrega un resultado sin justificarlo. En un sistema experto, el conocimiento está separado del mecanismo de inferencia, se puede actualizar de forma independiente y el sistema puede responder por qué llegó a cada conclusión. La siguiente tabla resume las diferencias.

Aspecto	Software convencional	ArquiSelect (sistema experto)
Conocimiento	Mezclado con el código en condicionales anidados	Separado en una base de conocimiento de hechos y reglas en Prolog
Mantenimiento	Cada cambio exige modificar el código del programa	Se agregan o modifican reglas sin tocar el motor de inferencia
Razonamiento	Flujo fijo y predefinido por el programador	Inferencia por encadenamiento y retroceso según los datos
Conflictos entre requisitos	Difíciles de modelar; se resuelven con orden de condicionales	Resueltos mediante pesos, prioridades y reglas de exclusión
Explicación	Entrega solo el resultado	Muestra las reglas que se activaron y por qué se descartaron alternativas
Información incompleta	Suele fallar o exigir todos los datos	Usa valores por defecto y ajusta el nivel de confianza

Objetivos del Proyecto

Objetivo General

Desarrollar un sistema experto, implementado en Prolog e integrado con Python, que recomiende el estilo de arquitectura de software más adecuado para un proyecto a partir de sus requisitos y restricciones, priorizando las alternativas y explicando el razonamiento que sustenta cada recomendación.

Objetivos Específicos

1. Representar en Prolog el conocimiento sobre estilos de arquitectura de software, atributos de calidad y características de los proyectos, mediante hechos y reglas.
2. Implementar un mecanismo de inferencia que priorice y ordene los estilos arquitectónicos candidatos según los requisitos y restricciones del proyecto.
3. Integrar la base de conocimiento con una interfaz en Python mediante pyswip, que permita ingresar los datos del proyecto y mostrar la recomendación con su explicación.

FUNDACIÓN ESCUELA TECNOLÓGICA DE NEIVA
JESÚS OVIEDO PÉREZ

Ficha de Definición del Proyecto y Pareja

Integrantes de la pareja

Nombre completo	Código
Nicolás Andrés Chica Córdoba	SOF220251010
Juan Carlos Bastidas Bernal	SOF20242010

Tema seleccionado

Título del sistema experto: ArquiSelect: Sistema Experto para la Selección de Estilos de Arquitectura de Software basado en Reglas Lógicas, Priorización de Atributos de Calidad y Explicación del Razonamiento, implementado en Prolog e integrado con Python.

Descripción del problema. La elección del estilo arquitectónico (monolito en capas, monolito modular, microservicios, serverless o arquitectura orientada a eventos) suele hacerse por intuición o por moda tecnológica. En equipos pequeños y con presupuesto limitado, como los de estudiantes y desarrolladores junior de Neiva y el Huila, esto genera sobrecostos de despliegue y mantenimiento, o sistemas que no soportan la demanda. El sistema recibe los requisitos y restricciones del proyecto, recomienda un estilo, ordena las alternativas por prioridad y explica el razonamiento.

Por qué aplica un sistema basado en reglas. El problema se resuelve con conocimiento experto expresable como reglas: requisitos que implican atributos de calidad, atributos que favorecen o penalizan estilos, y restricciones que excluyen estilos incompatibles. La decisión se obtiene por encadenamiento en tres niveles (validación, inferencia intermedia y conclusión), maneja conflictos entre requisitos mediante pesos y prioridades, y puede justificar cada resultado. Un programa convencional mezclaría estas condiciones dentro del código y no podría explicarlas ni actualizarlas sin modificarlo.

Listado preliminar de diagnósticos y resultados esperados:

- **Monolito en capas:** favorecido en proyectos con equipo pequeño, presupuesto bajo y entrega rápida.
- **Monolito modular:** favorecido en dominios complejos con poca necesidad de escalado independiente; es además el resultado por defecto.
- **Microservicios:** favorecido en alta demanda, alta disponibilidad y despliegue independiente; descartado con equipo pequeño o sin experiencia en DevOps.
- **Serverless:** favorecido en carga variable y presupuesto ajustado; descartado con procesos largos o latencia crítica.
- **Arquitectura orientada a eventos:** favorecida en integración asíncrona y alta disponibilidad; descartada si se exige consistencia estricta.
- **Prioridad y riesgo:** cada estilo viable sale con prioridad alta, media o baja y con riesgo bajo, medio o alto. Con una lista vacía el sistema devuelve monolito_modular con prioridad por_defecto.

3. Diccionario de Predicados

Modos: **+** entrada enviada desde Python, **-** salida hacia Python, **?** ambos. Python solo invoca el predicado principal y las consultas directas de las pruebas; el resto lo llama Prolog internamente.

Firma	Modo de instanciación	Descripción funcional	Desde Python
`diagnosticar_arquitectura/9`	+Reqs, -Mejor, -Prioridad, -Ranking, -Descartados, -Razones, -Perfil, -Desconocidos, -Suficiencia	Predicado principal: valida la lista, detecta el perfil, calcula el ranking con prioridad y riesgo, lista los estilos descartados y explica la recomendación.	motor.diagnosticar()
`filtrar_validos/2`	+Lista, -Validos	Conserva solo los requisitos reconocidos. El corte separa la cláusula que acepta de la que descarta.	Interno
`desconocidos/2`	+Lista, -Desc	Devuelve los requisitos enviados que no están en la base.	Interno
`requisito_conocido/1`	+R	Verdadero si R aparece en requisito_categoria/2.	Interno
`contiene_todos/2`	+Sub, +Lista	Verdadero si todos los elementos de Sub están en Lista (subconjunto).	motor.consultar()
`atributo_requerido/2`	+Reqs, ?Atributo	Enumera los atributos de calidad que exigen los requisitos.	Interno
`atributos_requeridos/2`	+Reqs, -Atributos	Lista ordenada y sin duplicados de atributos exigidos; [] si no hay ninguno.	Interno
`suficiencia/2`	+Validos, -Estado	Clasifica los datos como vacia, insuficiente (1 requisito) o suficiente.	Interno
`motivo_descarte/3`	+Estilo, +Reqs, ?Motivo	Encuentra un requisito incompatible con el estilo.	Interno
`estilo_viable/2`	?Estilo, +Reqs	Verdadero si el estilo no tiene ningún motivo de descarte (\+).	Interno
`aporte_total/3`	+Estilo, +Reqs, -Total	Suma los pesos de los atributos exigidos que el estilo favorece.	Interno
`penalizacion_total/3`	+Estilo, +Reqs, -Total	Suma las penalizaciones blandas causadas por los requisitos.	Interno
`puntaje/3`	+Estilo, +Reqs, -Puntaje	Aporte menos penalización.	Interno
`cubiertos/4`	+Estilo, +Reqs, -Cubiertos, -Cantidad	Filtra y cuenta los atributos exigidos que el estilo favorece.	motor.consultar()
`nivel_prioridad/2`	+Puntaje, -Nivel	Asigna alta, media o baja según los umbrales; la última cláusula es el valor por defecto.	Interno
`nivel_riesgo/3`	+Estilo, +Reqs, -Riesgo	Asigna riesgo bajo, medio o alto según la penalización acumulada.	Interno
`perfil_proyecto/2`	+Reqs, -Perfil	Primer perfil (alta_demanda, startup, empresarial) cuyos requisitos estén todos presentes; si no, general.	Interno
`recomendar/5`	?Estilo, +Reqs, -Puntaje, -Prioridad, -Riesgo	Estilo viable con su puntaje, prioridad y riesgo (conclusión de nivel 3).	Interno
`ranking/2`	+Reqs, -Ranking	Lista [Estilo, Puntaje, Prioridad, Riesgo] ordenada de mejor a peor, sin duplicados (setof).	Interno
`mejor_estilo/3`	+Reqs, -Estilo, -Prioridad	Primer estilo del ranking con puntaje positivo; si no hay, monolito_modular con prioridad por_defecto.	Interno
`descartados/2`	+Reqs, -Lista	Lista [Estilo, Requisito] de estilos excluidos por incompatibilidad.	Interno
`razones/3`	+Estilo, +Reqs, -Razones	Frases que explican qué atributos favorece el estilo y qué lo penaliza.	Interno

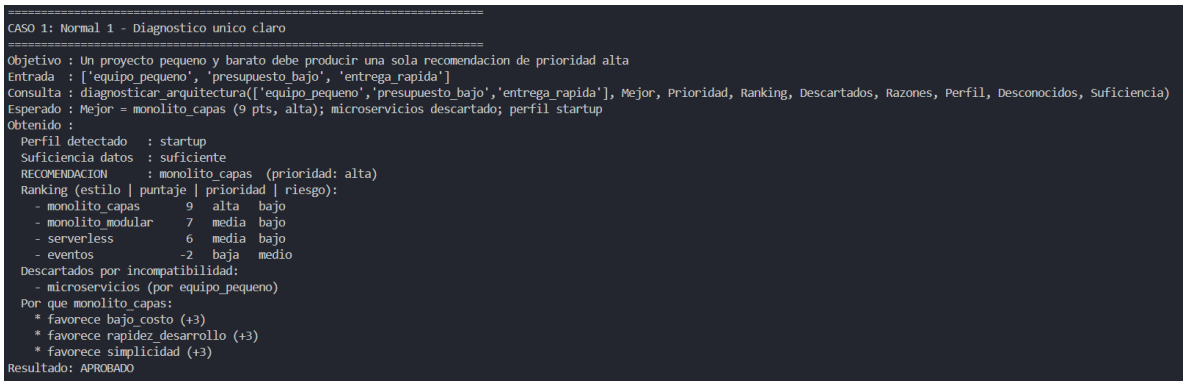
FUNDACIÓN ESCUELA TECNOLÓGICA DE NEIVA
JESÚS OVIEDO PÉREZ

Evidencia de Pruebas por Consola

Las pruebas se ejecutan con python3 pruebas.py. La salida completa (objetivo, entrada, consulta, esperado y obtenido) está en evidencia_pruebas.txt. Resultado de la ejecución: 6 de 6 casos aprobados.

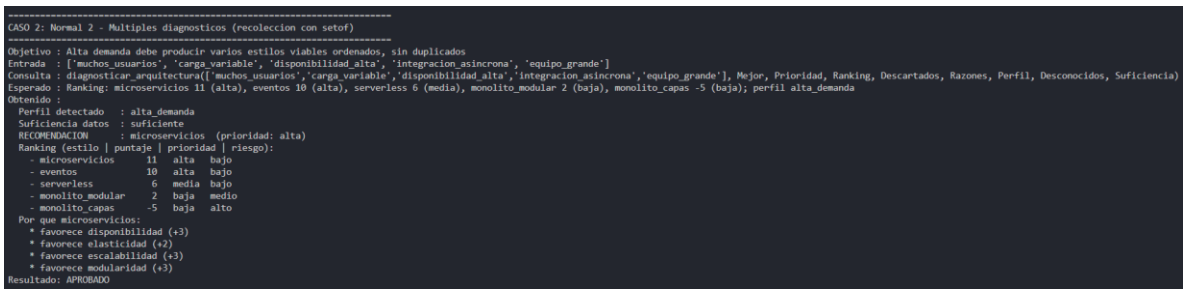
Caso	Entrada	Resultado esperado y obtenido
1. Normal, diagnóstico único claro	equipo_pequeno, presupuesto_bajo, entrega_rapida	monolito_capas (9 pts, alta); microservicios descartados; perfil startup
2. Normal, múltiples diagnósticos	muchos_usuarios, carga_variable, disponibilidad_alta, integracion_asincrona, equipo_grande	Ranking: microservicios 11, eventos 10, serverless 6, monolito_modular 2, monolito_capas -5
3. Exclusión (\+ y !)	equipo_pequeno, muchos_usuarios, disponibilidad_alta, procesos_largos	Descartados microservicios y serverless; mejor: eventos (3 pts, baja)
4. Borde, datos insuficientes	presupuesto_bajo, requisito_inventado, otro_desconocido	Suficiencia insuficiente; dos requisitos ignorados; monolito_capas tentativo (baja)
5. Borde, lista vacía	[ ]	monolito_modular, prioridad por_defecto, sin errores en Python
6. Listas directas	contiene_todos/2 y cubiertos/4	Subconjunto verdadero y falso; cubiertos = 3 atributos

Caso 1:



```
======================================================================
CASO 1: Normal 1 - Diagnostico unico claro
======================================================================
Objetivo : Un proyecto pequeno y barato debe producir una sola recomendacion de prioridad alta
Entrada  : ['equipo_pequeno', 'presupuesto_bajo', 'entrega_rapida']
Consulta : diagnosticar_arquitectura(['equipo_pequeno','presupuesto_bajo','entrega_rapida'], Mejor, Prioridad, Ranking, Descartados, Razones, Perfil, Desconocidos, Suficiencia)
Esperado : Mejor = monolito_capas (9 pts, alta); microservicios descartado; perfil startup
Obtenido :
  Perfil detectado   : startup
  Suficiencia datos  : suficiente
  RECOMENDACION      : monolito_capas  (prioridad: alta)
  Ranking (estilo | puntaje | prioridad | riesgo):
    - monolito_capas       9   alta   bajo
    - monolito_modular     7   media  bajo
    - serverless           6   media  bajo
    - eventos             -2   baja   medio
  Descartados por incompatibilidad:
    - microservicios (por equipo_pequeno)
  Por que monolito_capas:
    * favorece bajo_costo (+3)
    * favorece rapidez_desarrollo (+3)
    * favorece simplicidad (+3)
Resultado: APROBADO
```

Caso 2:



```
======================================================================
CASO 2: Normal 2 - Multiples diagnosticos (recoleccion con setof)
======================================================================
Objetivo : Alta demanda debe producir varios estilos viables ordenados, sin duplicados
Entrada  : ['muchos_usuarios', 'carga_variable', 'disponibilidad_alta', 'integracion_asincrona', 'equipo_grande']
Consulta : diagnosticar_arquitectura(['muchos_usuarios','carga_variable','disponibilidad_alta','integracion_asincrona','equipo_grande'], Mejor, Prioridad, Ranking, Descartados, Razones, Perfil, Desconocidos, Suficiencia)
Esperado : Ranking: microservicios 11 (alta), eventos 10 (alta), serverless 6 (media), monolito_modular 2 (baja), monolito_capas -5 (baja); perfil alta_demanda
Obtenido :
  Perfil detectado   : alta_demanda
  Suficiencia datos  : suficiente
  RECOMENDACION      : microservicios  (prioridad: alta)
  Ranking (estilo | puntaje | prioridad | riesgo):
    - microservicios      11   alta   bajo
    - eventos             10   alta   bajo
    - serverless           6   media  bajo
    - monolito_modular     2   baja   medio
    - monolito_capas      -5   baja   alto
  Por que microservicios:
    * favorece disponibilidad (+3)
    * favorece elasticidad (+2)
    * favorece escalabilidad (+3)
    * favorece modularidad (+3)
Resultado: APROBADO
```

Caso 3:

```
======================================================================
CASO 3: Exclusion (\+ y !)
======================================================================
Objetivo : Estilos incompatibles deben quedar fuera del ranking y aparecer como descartados
Entrada  : ['equipo_pequeno', 'muchos_usuarios', 'disponibilidad_alta', 'procesos_largos']
Consulta : diagnosticar_arquitectura(['equipo_pequeno','muchos_usuarios','disponibilidad_alta','procesos_largos'], Mejor, Prioridad, Ranking, Descartados, Razones, Perfil, Desconocidos, Suficiencia)
Esperado : Descartados: microservicios (equipo_pequeno) y serverless (procesos_largos); ninguno en el ranking; Mejor = eventos (3 pts, baja)
Obtenido :
  Perfil detectado   : alta_demanda
  Suficiencia datos  : suficiente
  RECOMENDACION      : eventos  (prioridad: baja)
  Ranking (estilo | puntaje | prioridad | riesgo):
    - eventos              3   baja   medio
    - monolito_modular     1   baja   medio
    - monolito_capas      -1   baja   alto
  Descartados por incompatibilidad:
    - microservicios (por equipo_pequeno)
    - serverless (por procesos_largos)
  Por que eventos:
    * favorece disponibilidad (+2)
    * favorece escalabilidad (+2)
    * penalizado por equipo_pequeno (-1)
Resultado: APROBADO
```

Caso 4:

```
======================================================================
CASO 4: Borde 1 - Datos insuficientes / lista parcial
======================================================================
Objetivo : Un solo requisito valido mas requisitos inventados: no debe fallar y debe avisar
Entrada  : ['presupuesto_bajo', 'requisito_inventado', 'otro_desconocido']
Consulta : diagnosticar_arquitectura(['presupuesto_bajo','requisito_inventado','otro_desconocido'], Mejor, Prioridad, Ranking, Descartados, Razones, Perfil, Desconocidos, Suficiencia)
Esperado : Suficiencia = insuficiente; desconocidos = [requisito_inventado, otro_desconocido]; recomendacion tentativa monolito_capas (3 pts, baja)
Obtenido :
  Perfil detectado   : general
  Suficiencia datos  : insuficiente
  Requisitos ignorados (no reconocidos): ['requisito_inventado', 'otro_desconocido']
  RECOMENDACION      : monolito_capas  (prioridad: baja)
  Ranking (estilo | puntaje | prioridad | riesgo):
    - monolito_capas       3   baja   bajo
    - monolito_modular     3   baja   bajo
    - serverless           2   baja   bajo
    - eventos              0   baja   bajo
    - microservicios      -2   baja   medio
  Por que monolito_capas:
    * favorece bajo_costo (+3)
Resultado: APROBADO
```

Caso 5:

```
======================================================================
CASO 5: Borde 2 - Lista vacia (valor por defecto)
======================================================================
Objetivo : Sin requisitos Prolog debe usar el fallback y Python no debe lanzar errores
Entrada  : []
Consulta : diagnosticar_arquitectura([], Mejor, Prioridad, Ranking, Descartados, Razones, Perfil, Desconocidos, Suficiencia)
Esperado : Mejor = monolito_modular, prioridad por_defecto, suficiencia vacia, ranking vacio
Obtenido :
  Perfil detectado   : general
  Suficiencia datos  : vacia
  RECOMENDACION      : monolito_modular  (prioridad: por_defecto)
Resultado: APROBADO
```

Caso 6:

```
======================================================================
CASO 6: Predicados de listas llamados directamente (subconjunto y conteo)
======================================================================
Objetivo : verificar contiene_todos/2 y cubiertos/4 sin pasar por el diagnostico
Consulta 1: contiene_todos([equipo_pequeno,presupuesto_bajo], ['presupuesto_bajo','equipo_pequeno','entrega_rapida'])
Esperado  : verdadero (1 solucion)
Obtenido  : verdadero
Consulta 2: contiene_todos([equipo_pequeno,muchos_usuarios], ['equipo_pequeno'])
Esperado  : falso (0 soluciones)
Obtenido  : falso
Consulta 3: cubiertos(serverless, ['muchos_usuarios','carga_variable','equipo_pequeno'], Cubiertos, Cantidad)
Esperado  : Cantidad = 3 (elasticidad, escalabilidad, simplicidad)
Obtenido  : Cubiertos = ['elasticidad', 'escalabilidad', 'simplicidad'], Cantidad = 3
Resultado : APROBADO
```